Software Requirements Specification

For Webhook Aggregator System (WACDP)

Version: 1.0 Baseline

Prepared by:

- **Sumedh Girish** (SRN: PES1UG24CS480)
- **Subramani B M** (SRN: PES1UG24CS473)
- **Sujan S Halanannavar** (SRN: PES1UG24CS477)
- **Skanda Shyam Nadig** (SRN: PES1UG24CS458)

Organization: Department of Computer Science & Engineering, PES University

Date Created: September 29, 2026

Project Code: BPS49 — Webhook Aggregator Core Development Pipeline

---

Revision History

Name	Date	Reason For Changes	Version
Team BPS49	September 20, 2026	Initial problem formulation, scope definition, and structural outline	0.1
Team BPS49	September 25, 2026	Added comprehensive Functional and Non-Functional Requirements	0.8
Team BPS49	September 29, 2026	Final IEEE Std 830 (1998) baseline release with UML Use Cases, Security section, and Traceability Matrix	1.0

Table of Contents

1. Introduction
- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions, Acronyms and Abbreviations
- 1.4 References
- 1.5 Overview

2. Overall Description
- 2.1 Product Perspective
- 2.1.1 System Interfaces
- 2.1.2 User Interfaces
- 2.1.3 Hardware Interfaces
- 2.1.4 Software Interfaces
- 2.1.5 Communications Interfaces

- 2.1.6 Memory Constraints
- 2.1.7 Operations
- 2.1.8 Site Adaptation Requirements
- 2.2 Product Functions
- 2.3 User Characteristics
- 2.4 Constraints
- 2.5 Assumptions and Dependencies
- 2.6 Apportioning of Requirements

3. Specific Requirements

- 3.1 External Interfaces
- 3.2 Functions (Functional Requirements FR-01 through FR-11)
- 3.3 Performance Requirements
- 3.4 Logical Database Requirements
- 3.5 Design Constraints
- 3.6 Software System Attributes
- 3.6.1 Reliability
- 3.6.2 Availability
- 3.6.3 Security (Objectives & Requirements)
- 3.6.4 Maintainability
- 3.6.5 Portability
- 3.7 UML Use Case Model & Detailed Use Cases
- 3.8 Additional Comments & Analysis

4. Supporting Information

- 4.1 Table of Contents and Index
- 4.2 Appendixes
- Appendix A: Requirements Traceability Matrix (RTM)
- Appendix B: Jira Scrum Epics & User Story Mapping

1. Introduction

The introduction provides an overview of the entire Software Requirements Specification (SRS) for the **Webhook Aggregator System (WACDP)**.

1.1 Purpose

The purpose of this document is to provide a complete, rigorous, and unambiguous specification of the requirements for the **Webhook Aggregator System (WACDP)**. This specification serves as the formal contract between stakeholders, system designers, developers, and quality assurance engineers. It details the functional capabilities, external interfaces, non-functional performance envelopes, security controls, and design constraints of the middleware.

Intended Audience:

- **Engineering Reviewers & Technical Architects:** To verify conformance to system requirements, scalability constraints, and engineering milestones.
- **System Architects:** To guide the derivation of the Software Architecture & Design Specification (IEEE Std 1016).
- **Development Team:** To direct sprint implementation across Jira backlog items (`WACDP-4` through `WACDP-11`).
- **QA & Verification Engineers:** To construct test plans and execution suites adhering to IEEE Std 829.

1.2 Scope

- **Software Produced:** Webhook Aggregator System (WACDP), a multi-threaded, asynchronous event ingestion, buffering, scheduling, and reliable delivery webhook middleware.
- **What the Software Will Do:**
- Ingest one-shot and recurring event payloads from upstream services with sub-15ms response latency.
- Buffer requests safely in memory (`service_queue`, default capacity 50,000 tasks, maximum 100,000 tasks) with active backpressure management.
- Categorize requests and allocate computational tasks to a pre-warmed worker thread pool (4 to 64 threads).
- Execute precision recurring timer callbacks using the host monotonic clock.
- Compute cryptographic HMAC-SHA256 signatures for message authenticity.
- Dispatch payloads to external HTTP/HTTPS target endpoints over mandatory TLS 1.3 with automated connection keep-alive.
- Implement an exponential backoff retry loop (`T_wait = 2^tries`) for transient destination failures up to `MAX_RETRIES = 5` (1 initial attempt + 5 retries = 6 total attempts).
- Provide reliable delivery with zero silent message loss by committing unrecoverable failures to a persistent Dead Letter Queue (DLQ) and append-only audit log.
- Expose administrative REST endpoints for status queries, health telemetry, and manual task redrives.
- **What the Software Will NOT Do:**
- It will not interpret or modify destination-specific internal business logic beyond envelope transformation.
- It will not function as a long-term arbitrary file-storage system.
- It will not provide a graphical user interface (GUI) desktop application; all interactions occur over RESTful APIs.
- **Benefits and Goals:** Decouples upstream microservices from downstream receiver availability, prevents cascading timeouts, cushions traffic spikes, and provides reliable asynchronous delivery with bounded retries and durable DLQ persistence.

1.3 Definitions, Acronyms and Abbreviations

- **WACDP:** Webhook Aggregator Core Development Pipeline.
- **SRS:** Software Requirements Specification.
- **DLQ:** Dead Letter Queue (durable storage for failed webhook payloads).
- **EDA:** Event-Driven Architecture.
- **FIFO:** First-In, First-Out queueing discipline.
- **HMAC:** Hash-based Message Authentication Code.
- **JWT:** JSON Web Token.
- **p99:** 99th percentile response time.
- **REST:** Representational State Transfer.
- **RFC:** Request for Comments (Internet Engineering Task Force standard).
- **RTM:** Requirements Traceability Matrix.
- **TLS:** Transport Layer Security (strictly TLS 1.3).
- **TSan:** ThreadSanitizer (dynamic race condition detector).
- **UUID:** Universally Unique Identifier.

1.4 References

1. **IEEE Std 830-1998:** *IEEE Recommended Practice for Software Requirements Specifications*.
2. **ISO/IEC/IEEE 29148:2018:** *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*.
3. **RFC 2119:** *Key words for use in RFCs to Indicate Requirement Levels*, Harvard University, March 1997.

4. **RFC 7230 / RFC 7231:** *Hypertext Transfer Protocol (HTTP/1.1): Message Syntax, Routing, and Semantics*, IETF, June 2014.
5. **RFC 8446:** *The Transport Layer Security (TLS) Protocol Version 1.3*, IETF, August 2018.
6. **WACDP Webhook Signing Specification:** *HMAC-SHA256 Egress Signature Scheme (inspired by GitHub & Stripe Webhook Security Standards)*, PES University, September 2026.
7. **Jira Scrum Board:** *Webhook Aggregator Core Development Pipeline (`WACDP`)*, Board ID 34, Sprints 1 & 2.

1.5 Overview

The remainder of this SRS is organized into three major sections:

- **Section 2 (Overall Description):** Describes the general factors that affect the product, including system perspective, user classes, operating environment, operational constraints, and assumptions.
- **Section 3 (Specific Requirements):** Contains the detailed functional requirements (`FR-01` through `FR-11`), performance metrics, logical database schema, software attributes (Reliability, Availability, Security, Maintainability, Portability), and the complete UML Use Case Model.
- **Section 4 (Supporting Information):** Contains the Requirements Traceability Matrix (Appendix A) and Jira Epics/Stories mapping (Appendix B).

---

2. Overall Description

2.1 Product Perspective

The Webhook Aggregator System is a self-contained, high-performance middleware service positioned between upstream event-producing microservices (e.g., Billing, Authentication, Order Management) and downstream third-party webhook receivers (e.g., Partner APIs, Slack, CRM endpoints). It replaces unmanaged, fragile point-to-point webhook delivery with a resilient, observable event pipeline.



2.1.1 System interfaces

- **Inbound Ingestion Interface:** HTTP REST API exposing `/api/v1/webhooks/ingest` and `/api/v1/webhooks/recurring`. Accepts JSON payloads from authenticated upstream microservices.
- **Outbound Dispatch Interface:** High-throughput HTTP/1.1 and HTTP/2 client establishing outbound TLS 1.3 socket connections to external target URLs.

- **Durable Persistence Interface:** Direct JDBC/ODBC or SQLite/RocksDB connection for write-ahead logging and DLQ table storage.
- **Administrative Telemetry Interface:** RESTful monitoring endpoint `/api/v1/health` and query endpoint `/api/v1/webhooks/{task_id}/status`.

2.1.2 User interfaces

- **API Client Interface:** Programmatic REST interface conforming strictly to OpenAPI 3.0 conventions.
- **Administrative CLI / Inspection Dashboard:** Provides formatted tabular status summaries of buffer queue depths, active worker threads, and dead-letter records with filtering capabilities.
- **Error Display:** Structured JSON error objects returning standard RFC 7807 problem details with error codes, descriptions, and retry intervals.

2.1.3 Hardware interfaces

- Standard x86_64 or ARM64 multi-core processor architecture (minimum 2 cores, recommended 8 cores for high-volume concurrency).
- Minimum 2 GB RAM (with 512 MB dedicated to buffer allocations).
- High-speed NVMe or SSD storage with write-ahead caching for persistent DLQ recording.

2.1.4 Software interfaces

- **Host Operating System:** POSIX-compliant operating system (Linux kernel 5.15+ with epoll, or macOS Darwin 22+ with kqueue).
- **Implementation Language & Toolchain:** Modern multi-threaded **C++20** conforming to ISO/IEC 14882:2020 standard, compiled with GCC 12+ or Clang 15+ using `-std=c++20`.
- **Persistent Storage Subsystem:** SQLite 3.35+ or PostgreSQL 15+ engine for durable DLQ transactions.
- **Cryptographic Library:** OpenSSL 3.0+ native platform crypto subsystem for HMAC-SHA256 signature calculations.

2.1.5 Communications interfaces

- **Network Protocol:** HTTP/1.1 and HTTP/2 over mandatory **TLS 1.3 only** (RFC 8446) for all ingress and egress channels. TLS 1.2 or earlier protocols are strictly rejected.
- **Data Serialization:** `application/json; charset=utf-8`.
- **Egress Headers:** Mandatory injection of `X-Webhook-ID`, `X-Webhook-Signature`, `X-Webhook-Timestamp`, and `Content-Type`.

2.1.6 Memory constraints

- Total resident set size (RSS) memory consumption MUST NOT exceed **500 MB** under a sustained load of 50,000 buffered tasks (and up to 100,000 tasks maximum). To satisfy this constraint while permitting maximum individual payloads up to 64 KB, the in-memory queue stores lightweight `WebhookTask` descriptors (~200 bytes containing `task_id`, `target_url`, timestamp, and a `PayloadReference`), while payloads exceeding 2 KB are staged in a high-speed local temporary payload store (memory-mapped NVMe offload buffer) and dereferenced on demand by worker threads.
- Static buffer memory allocations are bounded at initialization; dynamic expansion must degrade gracefully with active backpressure before hitting host OOM thresholds.

2.1.7 Operations

- **Continuous Unattended Mode:** The system runs as a long-running system daemon (systemd service or Docker container) without requiring human intervention for routine dispatching.
- **Automatic Recovery:** On host restart, persistent SQLite DLQ records and recurring schedules in the `recurring_schedules` table are preserved; active recurring cron schedules are re-armed automatically upon daemon startup.
- **Backup & Archival:** DLQ failure logs rotate daily and support automated archiving to compressed cold storage.

2.1.8 Site adaptation requirements

- Configurable environment variables for port bindings (`PORT=8080`), default queue buffer capacity (`DEFAULT_BUFFER_CAPACITY=50000`), maximum dynamic queue capacity (`MAX_BUFFER_CAPACITY=100000`), maximum retries (`MAX_RETRIES=5`, representing 1 initial attempt + 5 retries = 6 total attempts), and base backoff time (`BASE_WAIT_SECONDS=1`).

2.2 Product functions

1. **Asynchronous Request Ingestion:** Accept one-shot and recurring task submissions over REST API and validate payload schemas in under 15 ms.
2. **Buffering & Concurrency Management:** Push validated requests to an internal bounded, thread-safe FIFO buffer (`service_queue`, default 50,000 tasks, max 100,000 tasks).
3. **Dynamic Task Allocation:** Assign tasks to idle worker threads in the worker thread pool (4 to 64 threads) based on request type.
4. **Scheduled Timer Engine:** Register recurring event handles, manage timer callbacks, and periodically trigger computed payload events.
5. **Payload Computation & Transformation:** Execute user-defined transformations, add timestamp headers, and sign payloads with HMAC-SHA256.
6. **Resilient HTTP Dispatch:** Transmit webhook payloads to downstream endpoints over mandatory TLS 1.3 with connection keep-alive.
7. **Exponential Backoff Retry Subsystem:** Automatically retry transient network failures (HTTP 429, 5xx, socket timeouts/refusals) using formula `T_wait = 2^tries` up to `MAX_RETRIES = 5` (1 initial attempt + 5 retries = 6 total attempts). Non-retryable client errors (HTTP 400, 401, 403, 404) are not retried and route immediately to DLQ.
8. **Dead Letter Queue (DLQ) & Append-Only Audit Logging:** Persist unrecoverable failures into durable storage with complete error diagnostics.
9. **Status Query & Administrative Inspection:** Expose endpoints for checking task statuses, viewing DLQ payloads, and issuing manual redrives.

2.3 User characteristics

- **API Client Application Developers:** Highly experienced engineers integrating microservices. They require low-latency ingestion, clear HTTP 202 acknowledgements, and cryptographic signature predictability.
- **DevOps / Site Reliability Engineers (SRE):** Technical professionals responsible for operational uptime. They require granular Prometheus-compatible metrics, health endpoints, and automated recovery runbooks.
- **Target Endpoint Administrators:** Third-party receiver teams that verify incoming webhooks using HMAC-SHA256 signatures and expect strict anti-replay protection.

2.4 Constraints

1. **Thread Safety & Race Freedom (NFR-09):** Under the defined concurrency test workload, zero ThreadSanitizer-detected data races, deadlocks, or worker starvation SHALL occur, and lock contention SHALL remain below 5%.
2. **Memory Bounding & Queue Scalability (NFR-07):** Internal buffers MUST be bounded to prevent Out-Of-Memory (OOM) crashes under upstream traffic spikes (default 50,000 tasks, maximum 100,000 tasks, maintaining total process RSS ≤ 500 MB).
3. **Non-Blocking Ingestion:** The ingestion event loop MUST NOT perform blocking network I/O or disk operations on the ingestion path.
4. **Idempotency Support:** WACDP SHALL provide a stable unique `X-Webhook-ID` across retries; target endpoints MAY use it for idempotent processing.
5. **Regulatory & Audit Integrity (SEC-REQ-04):** Payload secrets (passwords, tokens, secret keys) must never be written to audit logs in plaintext; recursive JSON-aware sanitization must be enforced.

6. **Implementation Technology:** The middleware implementation language is strictly standardized on **C++20**.

2.5 Assumptions and dependencies

- Upstream API clients provide well-formed JSON payloads and valid authentication credentials.
- Target endpoints expose standard HTTP/HTTPS listeners and adhere to RFC 7231 status code conventions.
- DNS resolution services remain available for target endpoint hostnames.
- Host clock monotonically advances without irregular backward time shifts exceeding 50 ms.

2.6 Apportioning of requirements

- **Version 1.0 (Current Release):** Core asynchronous ingestion, thread-safe buffering, worker pool allocation, recurring timers, exponential backoff retries, persistent DLQ, and status inspection APIs.
- **Version 2.0 (Future Enhancement):** Distributed multi-node clustering with Raft consensus, dynamic distributed rate-limiting via Redis, and Kafka partition bridging.

---

3. Specific Requirements

3.1 External interfaces

3.1.1 Inbound Ingestion Request Specification

- **Item Name:** Webhook Ingestion Payload
- **Purpose:** Submit an event for asynchronous delivery.
- **Source:** Upstream API Client
- **Destination:** Ingestion Gateway Event Loop
- **Format:** JSON object conforming to the schema below:
- `type` (string, required): Enum `["one-shot", "recurring"]`.
- `target_url` (string, required): Fully qualified HTTP/HTTPS URL.
- `payload` (object, required): Arbitrary JSON payload data (maximum size 64 KB; requests exceeding 64 KB are rejected with HTTP 413 Payload Too Large; payloads > 2 KB are staged to local temporary storage).
- `schedule` (string or integer, optional): Cron string (e.g. `"0 */2 * * *"`) or interval in milliseconds (min `1000`).
- **Timing:** Ingestion response returned within 15 ms at p99.

3.1.2 Outbound Webhook Delivery Specification

- **Item Name:** Dispatched Webhook Request
- **Purpose:** Transmit the computed payload to the recipient endpoint.
- **Destination:** Target Endpoint
- **Headers Transmitted:**
- `Content-Type: application/json`
- `X-Webhook-ID: <UUIDv4>`
- `X-Webhook-Timestamp: <UnixEpochSeconds>`
- `X-Webhook-Signature: t=<Timestamp>,v1=<HMAC-SHA256-Hex>`
- **Response Expected:** HTTP status 200, 201, 202, or 204 within 5,000 ms.

3.2 Functions

In strict adherence to IEEE Std 830 specification standards, all functional requirements are specified using the **exact method of specification**:

FR-01: Asynchronous Request Ingestion & Schema Validation

- **Jira Mapping:** `WACDP-4` (EPIC-1: Request Ingestion Engine)
- **Description:** The system SHALL provide an HTTP POST endpoint (`/api/v1/webhooks/ingest`) that receives incoming webhook requests, validates the JSON schema, stages large payloads (> 2 KB) to temporary local storage, and acknowledges receipt immediately.
- **Inputs:** HTTP POST request containing `target_url` (valid URL), `payload` (JSON object up to 64 KB), `type` ("one-shot" or "recurring"), and optional `schedule` (cron string or interval seconds).
- **Processing Logic:**

1. Parse request body and authenticate client token.
2. Validate mandatory fields (`target_url`, `payload`, `type`).
3. Reject payloads exceeding 64 KB with HTTP 413 Payload Too Large; reject malformed JSON with HTTP 400 Bad Request.
4. Generate a unique `task_id` (UUIDv4) and timestamp.
5. Push the validated task descriptor into `service_queue`.
6. Return HTTP 202 Accepted with task metadata.

- **Outputs:** HTTP 202 Accepted response containing `{"task_id": "...", "status": "QUEUED", "timestamp": "..."}`.
- **Acceptance Criteria:** Syntactically valid requests achieve a 99th percentile response time (p99 latency) < 15 ms under a 1,000 req/sec load (with median p50 ≤ 3 ms).
- **Priority:** High (Mandatory)

FR-02: Non-Blocking Buffer Enqueueing (`service_queue`)

- **Jira Mapping:** `WACDP-4` (EPIC-1: Request Ingestion Engine)
- **Description:** The system SHALL enqueue ingested task descriptors into an internal thread-safe in-memory buffer (`service_queue`) without blocking the event loop. The queue operates with a default capacity of 50,000 tasks and a maximum dynamic capacity of 100,000 tasks. The warning/throttle threshold is 90% of current capacity, and the rejection threshold is 100% of current hard capacity.
- **Inputs:** Lightweight task descriptor containing `task_id`, `type`, `target_url`, `payload_ref`, `auth_headers`, and `created_at`.
- **Processing Logic:**

1. Acquire queue mutex or atomic pointer in non-blocking mode.
2. Inspect current queue depth against thresholds:

- Warning threshold: Depth reaches 90% of current capacity → log warning alert and trigger dynamic buffer expansion up to 100,000 tasks.
- Rejection threshold: Depth reaches 100% of current hard capacity (50,000 default or 100,000 max burst) → immediately return HTTP 429 Too Many Requests with `Retry-After: 5`.

3. Enqueue task descriptor (~200 bytes) and signal waiting worker threads via condition variable.

- **Outputs:** Queue size increment, worker wakeup signal.
- **Acceptance Criteria:** Enqueue operation completes in ≤ 50 µs; total process RSS memory remains ≤ 500 MB under 50,000 queued tasks; zero deadlocks observed under 100 concurrent producer threads.
- **Priority:** High (Mandatory)

FR-03: Request Categorization & Task Allocation

- **Jira Mapping:** `WACDP-5` (EPIC-2: Dispatcher & Retry Subsystem)
- **Description:** The worker thread pool SHALL pop requests from `service_queue`, categorize the request based on `type`, and branch execution into either immediate one-shot processing or recurring schedule registration.

- **Inputs:** Task object popped from `service_queue`.
- **Processing Logic:**

1. Worker thread pops next task from `service_queue`.
2. Inspect task attribute `type`:

- If `type == "one-shot"`: route to one-shot compute and dispatch pipeline.
- If `type == "recurring"`: route to recurring scheduler initialization.
- If unrecognized: route to failure audit log with status `INVALID_TYPE`.
- **Outputs:** Routed task instance in designated processing subsystem.
- **Acceptance Criteria:** Every popped task is categorized within 1 ms of queue departure; no task remains unallocated.
- **Priority:** High (Mandatory)

FR-04: One-Shot Request Execution & Dispatch Queueing

- **Jira Mapping:** `WACDP-5` (EPIC-2: Dispatcher & Retry Subsystem)
- **Description:** For one-shot tasks, the worker thread SHALL execute any required computational transformations, attach egress security headers, and enqueue the prepared payload into `dispatch_queue`.
- **Inputs:** One-shot task object.
- **Processing Logic:**

1. Format target payload and inject metadata (`webhook_id`, `dispatched_at`).
2. Compute HMAC-SHA256 signature using the registered endpoint secret key.
3. Construct dispatch package containing target URL, headers, and signed payload.
4. Push dispatch package to `dispatch_queue`.

- **Outputs:** Dispatch package placed on `dispatch_queue`.
- **Acceptance Criteria:** Transformation and signing complete in under 5 ms; payload accurately deposited in `dispatch_queue`.
- **Priority:** High (Mandatory)

FR-05: Scheduled Recurring Request Configuration

- **Jira Mapping:** `WACDP-6` (EPIC-2: Dispatcher & Retry Subsystem)
- **Description:** For recurring tasks, the system SHALL initialize a precision timer handle, store configuration parameters, and register the handle in the scheduler mapping table.
- **Inputs:** Recurring task object with schedule expression (`interval_ms` or cron format) and endpoint configuration.
- **Processing Logic:**

1. Parse cron expression or millisecond interval.
2. Instantiate timer handle with callback reference to task execution routine.
3. Store handle in thread-safe `scheduler_mapping` indexed by `schedule_id`.
4. Arm timer with host clock subsystem.

- **Outputs:** Active timer handle registered in `scheduler_mapping`.
- **Acceptance Criteria:** Timer handles successfully registered within 2 ms; scheduling precision within ± 10 ms of target trigger time.
- **Priority:** High (Mandatory)

FR-06: Timer-Driven Execution Callback Handling

- **Jira Mapping:** `WACDP-6` (EPIC-2: Dispatcher & Retry Subsystem)
- **Description:** Upon timer expiry event, the scheduler SHALL execute the registered callback, recalculate dynamic payload values, re-arm the timer handle, and push the generated payload to `dispatch_queue`.
- **Inputs:** Host timer expiry interrupt / callback trigger.
- **Processing Logic:**

1. Intercept timer event callback.
2. Retrieve task definition from `scheduler_mapping`.

3. Compute updated execution payload and timestamps.
4. Push computed payload into `dispatch_queue`.
5. Calculate next execution epoch and update timer handle.

- **Outputs:** Executed payload on `dispatch_queue`, timer re-armed.
- **Acceptance Criteria:** Zero missed timer firings during 24-hour continuous execution test with 500 active schedules.
- **Priority:** High (Mandatory)

FR-07: Endpoint Dispatch & Network Transmission

- **Jira Mapping:** `WACDP-7` (EPIC-2: Dispatcher & Retry Subsystem)
- **Description:** The dispatcher thread SHALL pop payloads from `dispatch_queue`, open an HTTP connection to the destination URL over mandatory TLS 1.3, and transmit the webhook payload.
- **Inputs:** Payload object popped from `dispatch_queue`.
- **Processing Logic:**

1. Pop payload from `dispatch_queue`.
2. Resolve target endpoint hostname and acquire HTTP connection from pool.
3. Issue HTTP POST request with timeout of 5,000 ms.
4. Inspect response code:

- If 200 ≤ status ≤ 299: Delivery successful. Safely release connection to pool.
- If status in {400, 401, 403, 404}: Non-retryable client error. Abort retries immediately and forward to Persistent Storage & Logger (FR-09).
- If status == 429, or 500 ≤ status ≤ 599, or connection refused / socket timeout: Transient error. Initiate exponential backoff retry loop (FR-08).
- **Outputs:** Delivery success log or transfer to retry loop / DLQ.
- **Acceptance Criteria:** Successful HTTP 2xx deliveries logged and closed cleanly; non-retryable 4xx errors routed directly to DLQ; transient failures handed over to retry loop without thread blockage.
- **Priority:** High (Mandatory)

FR-08: Exponential Backoff & Retry Mechanism

- **Jira Mapping:** `WACDP-7` (EPIC-2: Dispatcher & Retry Subsystem)
- **Description:** When an endpoint connection fails or responds with a retryable transient error (HTTP 429, 5xx, or network timeout/refusal), the system SHALL execute an exponential backoff retry loop up to `MAX_RETRIES = 5` retries (1 initial attempt + 5 retries = 6 total attempts). Non-retryable errors (HTTP 400, 401, 403, 404) SHALL NOT be retried.
- **Inputs:** Failed dispatch item, initial state (`tries = 0`, `wait_time = 1` second).
- **Processing Logic:**

1. If `tries < MAX_RETRIES` (where `MAX_RETRIES = 5`):

a. Increment `tries = tries + 1`.

b. Sleep/pause for `wait_time` seconds (asynchronously or non-blocking).

c. Double `wait_time = wait_time * 2`.

d. Attempt to send data payload to target endpoint.

e. If delivery succeeds (HTTP 2xx): safely close connection and terminate retry loop.

f. If delivery fails with non-retryable status (400, 401, 403, 404): terminate retry loop and forward immediately to Persistent Storage & Logger.

g. If delivery fails with transient status (429, 5xx, timeout): repeat loop.

2. If `tries >= MAX_RETRIES` (6 total attempts exhausted: 1 initial attempt + 5 retries) or initial connection cannot be established:

a. Exit retry loop.

b. Forward failed payload, error code, and retry history to Persistent Storage & Logger.

- **Outputs:** Retry attempt records, final transition to DLQ on exhaustion.

- **Acceptance Criteria:** Retry delays strictly follow sequence 1s, 2s, 4s, 8s, 16s (± 100 ms); total attempts strictly capped at 1 initial + 5 retries = 6 total attempts.
- **Priority:** High (Mandatory)

FR-09: Persistent Dead-Letter Storage & Failure Audit Logging

- **Jira Mapping:** `WACDP-8`, `WACDP-11` (EPIC-3: Monitoring, Audit & Persistence Subsystem)
- **Description:** The system SHALL persist all permanently failed webhook payloads (exhausted after 6 total attempts or non-retryable 4xx errors), metadata, and error stack traces into durable storage and record an append-only audit entry.
- **Inputs:** Exhausted or non-retryable task payload, HTTP status codes, error reason, retry timestamp log, destination URL.
- **Processing Logic:**

1. Construct Dead Letter Record (DLQ) with `task_id`, `target_url`, `payload`, `attempt_count`, `last_error`, and `failure_timestamp`.
2. Persist record into durable storage (ACID transaction).
3. Write formatted JSON log entry to append-only audit log file.

- **Outputs:** Durable DLQ database record and structured append-only audit log entry.
- **Acceptance Criteria:** 100% of payloads failing `MAX_RETRIES` or non-retryable 4xx are saved to persistent storage; zero data loss during simulated target endpoint outages.
- **Priority:** High (Mandatory)

FR-10: Status Query & Health Inspection API

- **Jira Mapping:** `WACDP-8` (EPIC-3: Monitoring, Audit & Persistence Subsystem)
- **Description:** The system SHALL provide REST endpoints for querying task delivery status by `task_id` and retrieving system health metrics.
- **Inputs:** HTTP GET `/api/v1/webhooks/{task_id}/status` or `/api/v1/health`.
- **Processing Logic:**

1. Authenticate requester.
2. Lookup `task_id` across in-memory active tables and persistent DLQ storage.
3. Aggregate metrics: current `service_queue` depth, active worker threads, total delivered, total failed.
4. Return JSON response.

- **Outputs:** HTTP 200 with status JSON (`QUEUED`, `PROCESSING`, `DELIVERED`, `RETRYING`, `FAILED`).
- **Acceptance Criteria:** Status query response latency ≤ 20 ms for 99% of requests.
- **Priority:** Medium (Desirable)

FR-11: Failed Request Inspection & Manual Redrive API

- **Jira Mapping:** `WACDP-8` (EPIC-3: Monitoring, Audit & Persistence Subsystem)
- **Description:** The system SHALL provide an administrative endpoint to list failed DLQ requests and trigger a manual redrive (re-enqueueing into `service_queue`).
- **Inputs:** HTTP GET `/api/v1/webhooks/failed` and HTTP POST `/api/v1/webhooks/{task_id}/retry`.
- **Processing Logic:**

1. Authenticate administrator identity.
2. Fetch failed records with pagination.
3. Upon POST retry request, reset `tries = 0`, update status to `QUEUED`, and push to `service_queue`.

- **Outputs:** Paginated list of failed webhooks; HTTP 200 on redrive success.
- **Acceptance Criteria:** Redriven task successfully re-enters the ingestion queue and follows standard dispatch pipeline.
- **Priority:** Medium (Desirable)

3.3 Performance requirements

- **Static Capacity Limits:**
- The system SHALL support buffering up to **100,000 tasks** in the internal `service_queue` without exhausting primary memory (maintaining total RSS ≤ 500 MB).
- The scheduler SHALL support tracking up to **1,000 concurrent recurring timers**.
- **Dynamic Numerical Requirements:**
- **Ingestion Latency (NFR-01 / WACDP-9):** Syntactically valid requests submitted to `/api/v1/webhooks/ingest` SHALL achieve a 99th percentile response time (p99 latency) < **15 milliseconds** (with median p50 ≤ 3 ms) under a concurrent load of 1,000 requests per second on reference 4-core, 8 GB RAM hardware.
- **Throughput Capacity (NFR-02 / WACDP-9):** The system SHALL sustain an ingestion rate of at least **5,000 requests per second** on reference 4-core, 8 GB RAM hardware.
- **Scheduler Precision (NFR-03):** Periodic timer callbacks SHALL execute within **± 10 milliseconds** jitter window of the targeted scheduled boundary.

3.4 Logical database requirements

The system maintains a relational or embedded persistent datastore (SQLite/PostgreSQL) for dead-letter persistence and recurring schedule state:

3.4.1 Dead Letter Table (`dead_letter_webhooks`)

- **Fields:**
- `task_id` (VARCHAR(36), Primary Key): UUIDv4 identifier.
- `target_url` (VARCHAR(512), NOT NULL): Webhook destination URL.
- `payload_body` (TEXT, NOT NULL): Sanitized JSON payload.
- `total_attempts` (INTEGER, NOT NULL): Number of dispatches executed (up to 1 initial + 5 retries = 6 total attempts).
- `last_error_code` (VARCHAR(64)): Final HTTP or network error code.
- `last_error_message` (TEXT): Diagnostic socket error or server response.
- `created_at` (TIMESTAMP WITH TIME ZONE): Ingestion timestamp.
- `failed_at` (TIMESTAMP WITH TIME ZONE): Terminal failure timestamp.
- `status` (VARCHAR(20)): Default `'DEAD_LETTER'`.
- `redrive_count` (INTEGER): Number of times manually re-queued (default 0).
- **Integrity Constraints & Indexing:** B-Tree index on `failed_at` and `status` to ensure fast paginated administrative retrieval.
- **Retention:** Records are retained for 30 days before automated archival.

3.4.2 Persistent Schedules Table (`recurring_schedules`)

- **Fields:**
- `schedule_id` (VARCHAR(36), Primary Key): UUIDv4 identifier.
- `target_url` (VARCHAR(512), NOT NULL): Webhook destination URL.
- `schedule_expr` (VARCHAR(64), NOT NULL): Cron string or interval in milliseconds.
- `payload_ref` (TEXT, NOT NULL): Serialized payload or offloaded store reference.
- `headers` (TEXT): Optional custom HTTP headers (JSON).
- `is_active` (BOOLEAN, NOT NULL DEFAULT 1): Active timer flag.
- `created_at` (TIMESTAMP WITH TIME ZONE): Creation timestamp.
- `updated_at` (TIMESTAMP WITH TIME ZONE): Modification timestamp.
- `last_triggered_at` (TIMESTAMP WITH TIME ZONE): Last callback execution timestamp.
- **Startup Recovery Flow:** On host/daemon restart, the scheduler automatically queries `recurring_schedules WHERE is_active = 1`, initializes corresponding `TimerHandle` instances, schedules the next monotonic execution epoch, and

registers them into memory without manual intervention.

3.5 Design constraints

- **Queue Scalability & Memory Bounding (NFR-07):** Internal queues enforce a default capacity of 50,000 tasks and a maximum dynamic capacity of 100,000 tasks with active HTTP 429 backpressure. Total process RSS memory MUST remain ≤ 500 MB by storing lightweight `WebhookTask` descriptors in RAM while staging raw payloads > 2 KB to local disk-backed temporary storage.
- **Worker Pool Elasticity & Socket Timeouts (NFR-08):** The worker thread pool dynamically scales between 4 core threads and 64 active threads based on queue backlog; outbound HTTP client connections enforce strict 5,000 ms connect and read timeouts to prevent socket starvation.
- **Thread Safety & Low Contention (NFR-09):** Under the defined concurrency test workload, zero ThreadSanitizer-detected data races, deadlocks, or worker starvation SHALL occur, and lock contention SHALL remain below 5%.
- **Zero Raw Pointers:** Memory management must be fully leak-free (validated via Valgrind/ASan).

3.6 Software system attributes

3.6.1 Reliability

- **Zero Silent Data Loss (NFR-04 / WACDP-11):** 100% of webhook requests accepted with HTTP 202 SHALL be either delivered successfully or committed transactionally to persistent DLQ storage.
- **Graceful Degradation & Backoff Sequence Fidelity (NFR-06):** Retries strictly adhere to the mathematical formula `T_wait = 2^tries` with absolute bounded jitter ≤ ± 100 ms up to `MAX_RETRIES = 5` (1 initial attempt + 5 retries = 6 total attempts). During downstream outages, the system absorbs tasks in the buffer queue and applies backpressure without crashing.

3.6.2 Availability

- **Service Availability (NFR-05):** The ingestion service SHALL maintain **99.95% availability** measured over a continuous 30-day operational observation window (allowing no more than 21.6 minutes of unplanned downtime per month).
- **Crash Recovery:** If the daemon process is restarted, persistent DLQ data remains uncorrupted, and active recurring schedules are reloaded from `recurring_schedules` within 3 seconds.

3.6.3 Security

In strict compliance with system security engineering specifications, the system establishes **3 Security Objectives** and **4 Security Requirements**:

Security Objectives:

1. **SEC-OBJ-01: In-Flight & Egress Payload Authenticity & Confidentiality**

The system MUST guarantee that payloads cannot be intercepted or modified in flight, and MUST provide cryptographic proof to target endpoints that webhooks originated exclusively from the Webhook Aggregator System.

2. **SEC-OBJ-02: Perimeter Protection & Non-Repudiation**

The system MUST prevent unauthorized entities from injecting arbitrary webhooks, consuming buffer memory, or spoofing caller identities, while maintaining an append-only audit log.

3. **SEC-OBJ-03: Resource Exhaustion & Denial-of-Service (DoS) Mitigation**

The system MUST protect internal worker pools, queue buffers, and downstream endpoints from memory exhaustion and flood attacks via adaptive rate limiting.

Security Requirements:

1. **SEC-REQ-01: Cryptographic Payload Signing via HMAC-SHA256**

For every outbound webhook dispatch, the system SHALL compute an HMAC using SHA-256 according to the WACDP Webhook Signing Specification:

```
Signature = HMAC-SHA256(SecretKey, Timestamp + "." + PayloadBody)
```

The hexadecimal digest MUST be transmitted in the `X-Webhook-Signature` header (`t=<Timestamp>,v1=<HexDigest>`) alongside `X-Webhook-Timestamp`. Target endpoints verify this signature and reject replay attacks older than 300 seconds.

2. SEC-REQ-02: Ingestion API Authentication and Rate Limiting

The `/api/v1/webhooks/ingest` endpoint SHALL require a Bearer token in the `Authorization` header. Requests lacking valid credentials MUST be rejected with HTTP 401 Unauthorized in under 2 ms. The gateway SHALL enforce a Token Bucket limit of **1,000 requests per minute per API key**, returning HTTP 429 Too Many Requests upon limit breach.

3. SEC-REQ-03: Mandatory Transport Layer Security (TLS 1.3 Only)

All inbound connections and outbound endpoint dispatches SHALL enforce **TLS 1.3 only** (RFC 8446) with secure cipher suites (`TLS_AES_256_GCM_SHA384`). Connection attempts utilizing TLS 1.2, TLS 1.1, or unencrypted plaintext HTTP MUST be strictly rejected.

4. SEC-REQ-04: Append-Only Audit Trail & JSON-Aware Sensitive Data Sanitization

The persistent DLQ and failure logger SHALL write audit entries to an append-only log. The logger MUST recursively traverse all JSON keys and values, detecting sensitive property names (`password`, `secret`, `token`, `api_key`, `authorization`, `access_token`, `client_secret`, `private_key`) across nested objects/arrays, and replace their values with `***REDACTED***` before writing records to disk.

3.6.4 Maintainability

- **Modularity (NFR-10):** Ingestion, scheduling, worker dispatching, and persistence components are decoupled via well-defined C++20 interfaces.
- **Code Hygiene:** Adheres to IEEE standard documentation and clean code practices with unit test coverage exceeding 85%.

3.6.5 Portability

- **Platform Independence (NFR-11):** The C++20 codebase compiles and runs without modification across POSIX environments including Ubuntu 22.04 LTS, RHEL 9, and macOS Darwin.
- **Containerization:** Distributed with an OCI-compliant Dockerfile for rapid deployment on Kubernetes or bare-metal Linux.
- **Standards Adherence (NFR-12):** All API communications strictly conform to RFC 7230, RFC 7231, and RFC 8446 (TLS 1.3). Egress signing adheres to the WACDP Webhook Signing Specification.

3.7 UML Use Case Model & Detailed Use Cases

3.7.1 UML Use Case Diagram

The following UML Use Case Diagram illustrates the system boundary, actors, core use cases, and relationship associations (`«include»`, `«extend»`):

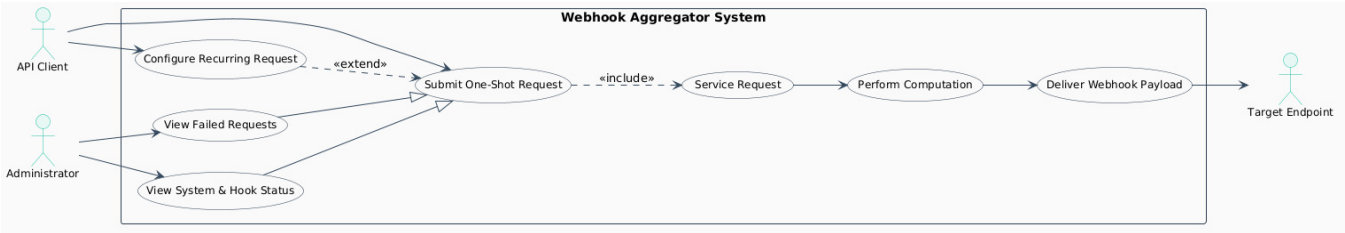
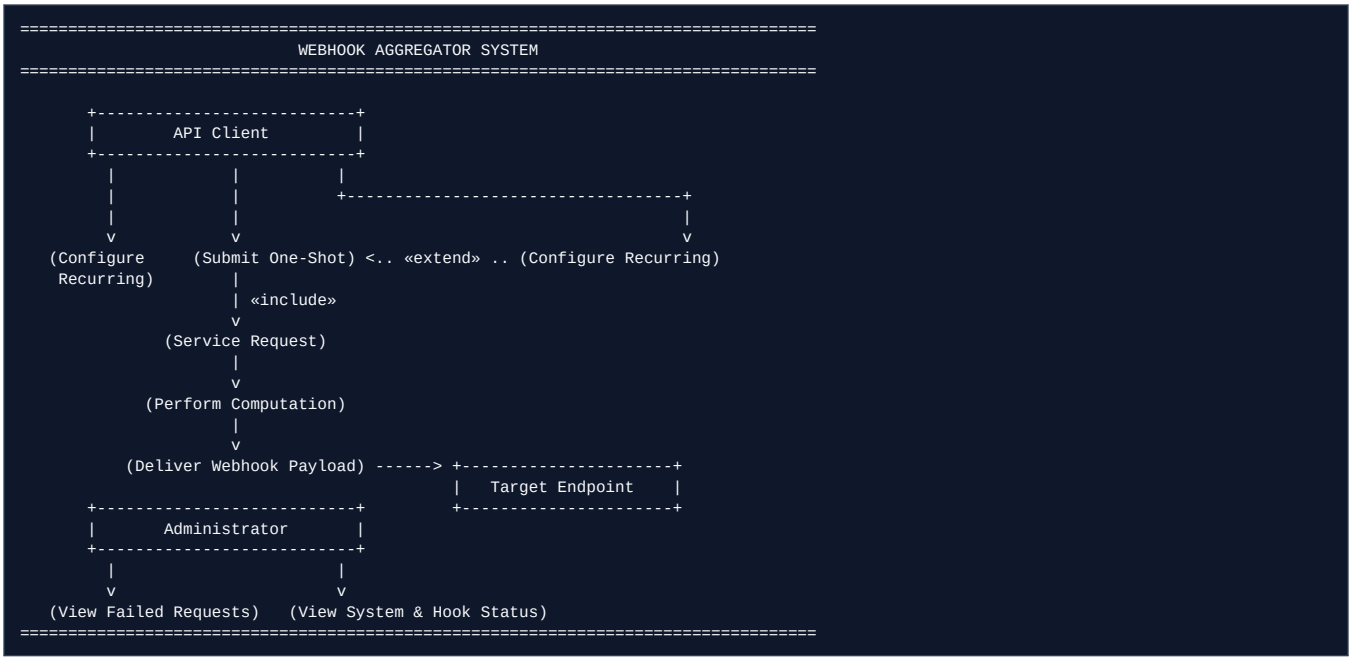


Figure: Webhook Aggregator System Use Case Diagram



3.7.2 Actor Definitions

1. **API Client (Primary Actor):** Upstream service that submits one-shot tasks and registers recurring delivery schedules.
2. **Administrator (Secondary Actor):** System engineer who inspects health metrics, monitors dead-letter queues, and triggers manual retries.
3. **Target Endpoint (External Actor):** Third-party receiver that receives and acknowledges dispatched webhook payloads.

3.7.3 Detailed Use Case Specifications

UC-01: Submit One-Shot Request

- **Primary Actor:** API Client
- **Preconditions:** Client holds a valid Bearer token; ingestion service is active.
- **Trigger:** Client sends HTTP POST to `/api/v1/webhooks/ingest` with `type: "one-shot"`.
- **Main Success Scenario:**

1. Client submits request with `target_url` and `payload`.
2. System validates JSON schema and authentication token.
3. System creates unique `task_id` and enqueues task into `service_queue`.
4. System returns HTTP 202 Accepted with `task_id` to API Client.
5. System services request asynchronously (includes UC-03).

- **Alternative Flows:**
- *2a. Schema validation failure:* Returns HTTP 400 Bad Request.
- *2b. Authentication failure:* Returns HTTP 401 Unauthorized.
- *3a. Queue saturated:* Applies backpressure and returns HTTP 429 Too Many Requests.
- **Postconditions:** Task is enqueued in `service_queue`; client holds unique `task_id`.

UC-02: Configure Recurring Request

- **Primary Actor:** API Client
- **Relationship:** Extends `UC-01` when scheduling parameters are provided.
- **Preconditions:** Client supplies a valid cron expression or millisecond interval (≥ 1,000 ms).
- **Trigger:** Client submits request with `type: "recurring"` and `schedule` configuration.
- **Main Success Scenario:**

1. Client submits recurring request specification.
2. System validates cron format and interval limits.
3. System initializes timer handle and registers it in `scheduler_mapping`.
4. System returns HTTP 201 Created with `schedule_id`.
5. On timer expiration, system generates recurring task instances that execute UC-01/UC-03.

- **Alternative Flows:**
- *2a. Invalid cron syntax:* Returns HTTP 422 Unprocessable Entity.
- **Postconditions:** Timer handle is armed in the host scheduler mapping table.

UC-03: Service Request

- **Primary Actor:** System (Worker Thread Pool)
- **Relationship:** Included by `UC-01`.
- **Preconditions:** At least one task is present in `service_queue`; worker thread is idle.
- **Trigger:** Worker thread wakes up on condition variable notification.
- **Main Success Scenario:**

1. Worker pops next task from `service_queue`.
2. Worker inspects task type (`one-shot` vs `recurring`).
3. Worker executes handler and invokes `UC-04: Perform Computation`.

- **Postconditions:** Task is actively processing on worker thread.

UC-04: Perform Computation

- **Primary Actor:** System (Worker Thread Pool)
- **Preconditions:** Task is allocated to active worker thread.
- **Trigger:** Invoked by `UC-03`.
- **Main Success Scenario:**

1. Worker parses raw task payload.
2. Worker computes timestamp tokens, dynamic parameters, and checksums.
3. Worker signs payload with endpoint HMAC-SHA256 signature.
4. Worker pushes prepared dispatch package into `dispatch_queue`.

- **Postconditions:** Formatted, signed dispatch item is enqueued in `dispatch_queue`.

UC-05: Deliver Webhook Payload

- **Primary Actor:** System (Dispatcher Thread)
- **Secondary Actor:** Target Endpoint
- **Preconditions:** Prepared dispatch item is available in `dispatch_queue`.
- **Trigger:** Dispatcher extracts item from `dispatch_queue`.
- **Main Success Scenario:**

1. Dispatcher initiates HTTP POST request to destination URL.
2. Target Endpoint responds with HTTP 200/201/204 within 5,000 ms.
3. Dispatcher records successful delivery timestamp and releases connection to pool.

- **Extensions (Retry Mechanism):**
- *2a. Target Endpoint returns 5xx error or connection times out:*

1. Dispatcher initiates retry loop (`tries = 0`, `wait_time = 1s`).
2. Dispatcher waits `wait_time` seconds.
3. Dispatcher retries transmission.
4. If successful: connection is closed cleanly; use case terminates.
5. If unsuccessful and `tries < MAX_TRIES`: increments `tries`, doubles `wait_time = wait_time * 2`, repeats step 2.

6. If `tries >= MAX_TRIES`: system aborts retry loop, records failed payload into persistent storage, and writes audit failure log.

- **Postconditions:** Payload is delivered to Target Endpoint OR permanently stored in DLQ.

UC-06: View Failed Requests

- **Primary Actor:** Administrator
- **Preconditions:** Administrator is authenticated.
- **Trigger:** Admin issues GET request to `/api/v1/webhooks/failed`.
- **Main Success Scenario:**

1. System queries persistent failure storage.
2. System returns paginated JSON list of failed webhooks with error codes and retry histories.
3. Admin inspects failed records and error stack traces.

- **Postconditions:** Failed records retrieved without altering stored state.

UC-07: View System & Hook Status

- **Primary Actor:** Administrator
- **Preconditions:** Ingestion system is active.
- **Trigger:** Admin issues GET request to `/api/v1/webhooks/{task_id}/status` or `/api/v1/health`.
- **Main Success Scenario:**

1. Admin requests status for specific `task_id` or global telemetry.
2. System compiles queue depths, worker states, and task delivery lifecycle stage.
3. System returns HTTP 200 and operational telemetry JSON.

- **Postconditions:** Accurate system metrics delivered to Administrator.

3.8 Additional comments & Analysis

The requirements in this specification follow a hybrid organization combining **Feature Organization (Section 3.7.4)** with **Functional Hierarchy (Section 3.7.7)** in accordance with IEEE Std 830-1998 guidelines. The system's high-throughput characteristics make the stimulus-response pairing of the two-stage queue buffering pattern ideal for formal verification.

---

4. Supporting Information

4.1 Table of Contents and Index

A complete Table of Contents is provided at the beginning of this document. Detailed index cross-references are maintained in Appendix A.

4.2 Appendixes

Appendix A: Requirements Traceability Matrix (RTM)

Requirement ID	Requirement Summary	Jira Issue	Architectural Component	Test Case ID
FR-01	Asynchronous Ingestion & Schema Validation	`WACDP-4`	Ingestion Gateway & Event Loop	`TC-FR-01`
FR-02	Non-Blocking Enqueueing (`service_queue`)	`WACDP-4`	In-Memory `service_queue` Buffer	`TC-FR-02`

Requirement ID	Requirement Summary	Jira Issue	Architectural Component	Test Case ID
FR-03	Request Categorization & Task Allocation	`WACDP-5`	Worker Thread Pool Manager	`TC-FR-03`
FR-04	One-Shot Request Execution & Dispatch Queue	`WACDP-5`	Worker Thread & `dispatch_queue`	`TC-FR-04`
FR-05	Recurring Schedule Configuration	`WACDP-6`	Scheduler & Timer Subsystem	`TC-FR-05`
FR-06	Timer Callback Handling & Execution	`WACDP-6`	Timer Engine & Callback Registry	`TC-FR-06`
FR-07	Endpoint Dispatch & Network Delivery	`WACDP-7`	Dispatcher Thread & Connection Pool	`TC-FR-07`
FR-08	Exponential Backoff Retry Loop	`WACDP-7`	Retry Subsystem Engine	`TC-FR-08`
FR-09	Persistent Dead-Letter Storage & Audit Logging	`WACDP-8, 11`	Persistent Storage (DLQ) & Logger	`TC-FR-09`
FR-10	Status Query & Health Inspection API	`WACDP-8`	Administrative Status Controller	`TC-FR-10`
FR-11	Failed Request Inspection & Manual Redrive	`WACDP-8`	DLQ Admin Controller	`TC-FR-11`
NFR-01	Ingestion Latency (p99 < 15 ms)	`WACDP-9`	Ingestion Gateway & Event Loop	`TC-NFR-01`
NFR-02	Sustained Throughput (5,000 req/s)	`WACDP-9`	Pipeline Architecture & Thread Pool	`TC-NFR-02`
NFR-03	Timer Precision (± 10 ms)	`WACDP-6`	Scheduler & Timer Subsystem	`TC-NFR-03`
NFR-04	Zero Silent Data Loss / Durability	`WACDP-11`	Persistent DLQ & Write-Ahead Store	`TC-NFR-04`
NFR-05	High Availability & Crash Recovery (99.95% Uptime / 30-Day Window)	`WACDP-4`	Gateway Architecture & State Re-arm	`TC-NFR-05`
NFR-06	Graceful Degradation & Backoff Sequence Fidelity (`T_wait = 2^tries` ± 100 ms)	`WACDP-7`	Dispatcher & Backoff Engine	`TC-NFR-06`
NFR-07	Queue Scalability & Memory Bounding (50k/100k, RSS ≤ 500 MB)	`WACDP-4`	Bounded `service_queue` & Disk Offload	`TC-NFR-07`
NFR-08	Worker Pool Elasticity & Socket Timeouts (4–64 Threads, 5,000 ms)	`WACDP-7`	Dynamic Worker Pool & Transport Client	`TC-NFR-08`
NFR-09	Thread Safety & Low Contention (Zero TSan Races, Contention < 5%)	`WACDP-10`	Concurrent Queues & Mutexes	`TC-NFR-09`
NFR-10	Subsystem Modularity (Decoupled C++20 Subsystems, >85% Coverage)	`WACDP-5`	Core Architecture Interfaces	`TC-NFR-10`
NFR-11	Multi-Platform POSIX Portability (Linux Ubuntu & macOS Darwin)	`WACDP-4`	Build Pipeline & Target OS	`TC-NFR-11`
NFR-12	Standards Adherence (RFC 7230/7231/8446 & WACDP Webhook Signing)	`WACDP-7`	Ingestion Gateway & Dispatcher	`TC-NFR-12`
SEC-REQ-01	HMAC-SHA256 Payload Signature Verification	`WACDP-7`	Worker Crypto Transform & Headers	`TC-SEC-01`
SEC-REQ-02	Ingestion Token Auth & Rate Limiting	`WACDP-4`	API Gateway Security Interceptor	`TC-SEC-02`
SEC-REQ-03	TLS 1.3 Transport Security	`WACDP-7`	Network Transport Client	`TC-SEC-03`

Requirement ID	Requirement Summary	Jira Issue	Architectural Component	Test Case ID
SEC-REQ-04	Append-Only Audit Log & Secret Redaction	WACDP-11	Failure Audit Logger & Sanitizer	TC-SEC-04

Appendix B: Jira Scrum Epics & User Story Mapping

The following table cross-references the requirements in this SRS with the user stories from the Jira project backlog (SCRUM_SumedhGirish_PES1UG24CS480_BPS49WebhookAggregator.pdf):

Jira Issue ID	Issue Summary	Sprint	Associated Epics	Related SRS Requirements
WACDP-4	Request Ingestion & Asynchronous Buffering	Sprint 1	EPIC-1: Request Ingestion Engine	FR-01, FR-02, NFR-05, NFR-07, NFR-11, SEC-REQ-02
WACDP-5	Request Categorization & Task Allocation	Sprint 2	EPIC-2: Dispatcher & Retry Subsystem	FR-03, FR-04, NFR-10
WACDP-6	Scheduled Timer Configuration & Execution	Sprint 2	EPIC-2: Dispatcher & Retry Subsystem	FR-05, FR-06, NFR-03
WACDP-7	Endpoint Dispatch & Exponential Backoff Retry	Sprint 2	EPIC-2: Dispatcher & Retry Subsystem	FR-07, FR-08, NFR-06, NFR-08, NFR-12, SEC-REQ-01, SEC-REQ-03
WACDP-8	Failure Audit Logging & Status Query API	Backlog	EPIC-3: Monitoring, Audit & Persistence	FR-09, FR-10, FR-11
WACDP-9	Low Ingestion Latency	Backlog	EPIC-1: Request Ingestion Engine	NFR-01, NFR-02
WACDP-10	Thread Safety & Concurrency	Backlog	EPIC-2: Dispatcher & Retry Subsystem	NFR-09
WACDP-11	Data Durability & Auditability	Backlog	EPIC-3: Monitoring, Audit & Persistence	NFR-04, SEC-REQ-04

End of Software Requirements Specification (SRS) — IEEE Std 830-1998